

Agenda: Indices Columnstore

- Arquitetura dos Índices Columnstore
- Comparando Columnstore x Btree
- Criando Índices Columnstore
- **Hands On:** Criando Índices Columnstore
- Desempenho em Atualizações



• Introdução ao Índice Columnstore

- ✓ Disponível a partir do SQL Server 2012.
- ✓ Melhora o desempenho de algumas consultas, comparando com Btree.
- ✓ Ganho no desempenho de consultas com grande volume de linhas de retorno e/ou com operações agregadas (GROUP BY).
- ✓ Tecnologia chamada **xVelocity**
 - Utiliza a memória para armazenar a estrutura do índice.
 - Índice organizado por coluna.
- ✓ **xVelocity** possui duas implementações no SQL Server:
 1. **xVelocity in-memory analytics engine**: índice do Analysis Service Tabular para análise de dados em Business Intelligence, chamado inicialmente de Vertipaq.
 2. **xVelocity memory optimized ColumnStore index**: mesma tecnologia usada no Analysis Service, disponibiliza índice em coluna (Column Store Index) no SQL Server Engine.



- **Limitações no uso dos Índices Columnstore**

- ✓ No SQL Server 2012 tabela com Índice Columnstore não pode receber atualizações.
- ✓ A partir do SQL Server 2014 tabelas podem receber atualizações.
- ✓ Baixo desempenho nas atualizações comparando com índice Btree.

- **Melhorias disponíveis a partir do SQL Server 2016**

- ✓ Nonclustered Columnstore Index pode conter cláusula de filtro.
- ✓ Tabelas in-memory podem ter Columnstore Index.
- ✓ Tabelas com Clustered Columnstore Index podem ter chave primária e chave estrangeira (no SQL 2014 não pode).



- **Arquitetura Índice Btree**

✓ 3 linhas por Data Page

<i>Saledate</i>	<i>Product</i>	<i>Amt</i>	<i>GrossPrice</i>	<i>SalesTax</i>	<i>NetPrice</i>	<i>...</i>
2012-03-08	Candy bar	50	75.00	14.25	89.25	...
2012-03-10	Smart phone	1	349.50	66.41	419.91	...
2012-03-11	Apple (bag)	7	31.57	1.89	33.46	...
2012-03-12	Smart phone	1	349.50	66.41	419.91	...
2012-03-19	Chair	1	599.50	113.91	713.41	...
2012-03-20	Toy car	3	29.97	5.69	35.66	...
2012-03-20	Chair	3	1,798.50	341.72	2,140.22	...
2012-03-20	Laptop	2	2,860.00	543.40	3,403.40	...
2012-03-21	Apple (bag)	14	63.14	3.79	66.93	...
2012-03-24	Pocket knife	1	12.95	2.46	15.41	...
2012-03-27	Apple (bag)	2	9.02	0.54	9.56	...
2012-03-28	Candy bar	5	7.50	1.43	8.93	...



- **Arquitetura Índice Columnstore**

- ✓ Data Page armazena dados de uma coluna

<i>Saledate</i>	<i>Product</i>	<i>Amt</i>	<i>GrossPrice</i>	<i>SalesTax</i>	<i>NetPrice</i>	<i>...</i>
2012-03-08	Candy bar	50	75.00	14.25	89.25	...
2012-03-10	Smart phone	1	349.50	66.41	419.91	...
2012-03-11	Apple (bag)	7	31.57	1.89	33.46	...
2012-03-12	Smart phone	1	349.50	66.41	419.91	...
2012-03-19	Chair	1	599.50	113.91	713.41	...
2012-03-20	Toy car	3	29.97	5.69	35.66	...
2012-03-20	Chair	3	1,798.50	341.72	2,140.22	...
2012-03-20	Laptop	2	2,860.00	543.40	3,403.40	...
2012-03-21	Apple (bag)	14	63.14	3.79	66.93	...
2012-03-24	Pocket knife	1	12.95	2.46	15.41	...
2012-03-27	Apple (bag)	2	9.02	0.54	9.56	...
2012-03-28	Candy bar	5	7.50	1.43	8.93	...



• Arquitetura Índice Columnstore

- ✓ Desenvolvido para tabelas grandes.
- ✓ Divide os dados em grupos de linhas (rowgroups), e cada coluna em segmentos de até 1 milhão de valores.

Segments (1 million values)

	<i>Saledate</i>	<i>Product</i>	<i>Amt</i>	<i>GrossPrice</i>	<i>SalesTax</i>	<i>NetPrice</i>	<i>...</i>
<i>Rowgroup (1 million rows)</i>	2012-03-08	Candy bar	50	75.00	14.25	89.25	...
	2012-03-10	Smart phone	1	349.50	66.41	419.91	...
	2012-03-11	Apple (bag)	7	31.57	1.89	33.46	...
	2012-03-12	Smart phone	1	349.50	66.41	419.91	...
	2012-03-19	Chair	1	599.50	113.91	713.41	...
	2012-03-20	Toy car	3	29.97	5.69	35.66	...
	2012-03-20	Chair	3	1,798.50	341.72	2,140.22	...
	2012-03-20	Laptop	2	2,860.00	543.40	3,403.40	...
	2012-03-21	Apple (bag)	14	63.14	3.79	66.93	...
	2012-03-24	Pocket knife	1	12.95	2.46	15.41	...
	2012-03-27	Apple (bag)	2	9.02	0.54	9.56	...
	2012-03-28	Candy bar	5	7.50	1.43	8.93	...



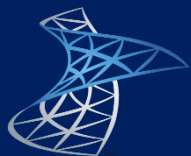
- **Arquitetura Índice Columnstore**

- ✓ Consulta

- O SQL Server armazena no metadata de cada segmento o menor e maior valor, para eliminação na consulta.
- Este processo de eliminação não funciona para tipos string.

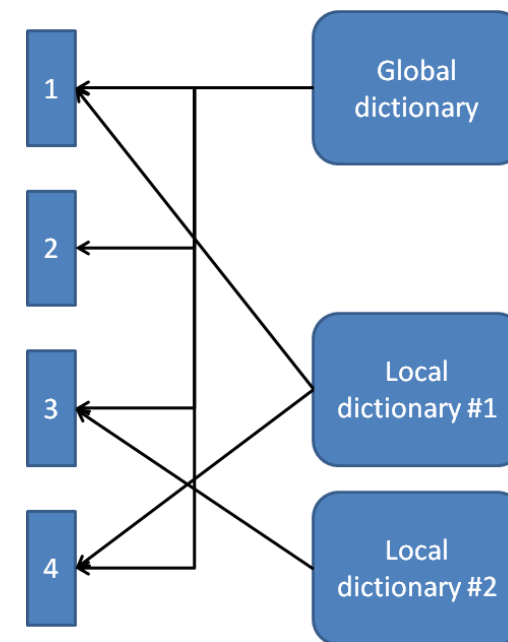
- ✓ Compressão

- Compressão muito eficiente, pois comprime dados das colunas com muito valor repetido.
- Exemplo:
 - Btree row compression: 44%
 - Btree page compression: 25%
 - Columnstore: 9.9%



• Arquitetura índice Columnstore

- ✓ Utiliza dicionário para tipos de dados string.
- ✓ SQL Server identifica todas as ocorrências e atribui um ID interno para cada ocorrência, por exemplo: (Black, 1); (Red, 2); (Green, 3); (Blue, 4).
- ✓ Este dicionário fica armazenado em espaço separado da tabela.
- ✓ Na tabela o SQL Server armazena o ID, no exemplo acima 1 para **Black**, 2 para **Red**, etc.
- ✓ Existe um dicionário global e outro local para cada segmento.





• Criando Índice Columnstore

✓ Índice Columnstore com Filtro

```
CREATE NONCLUSTERED COLUMNSTORE INDEX ixc_SalesOrderHeader_OrderDate  
ON dbo.SalesOrderHeader (OrderDate, CustomerID, SubTotal)  
WHERE OrderDate < '20060101'
```

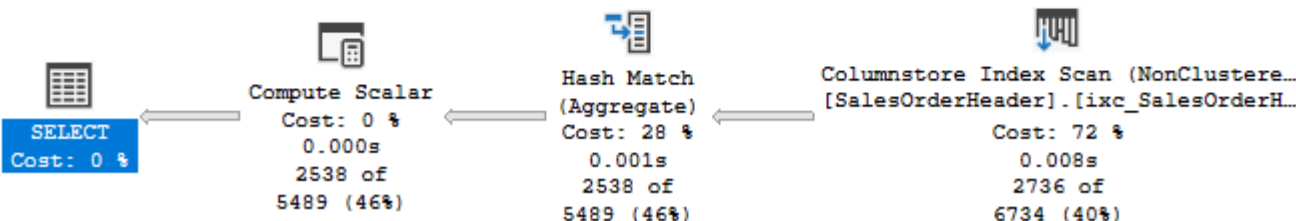
```
SELECT CustomerID, sum(SubTotal) CustomerTotal, count(*) OrderQt  
FROM dbo.SalesOrderHeader  
WHERE OrderDate >= '20020401' AND OrderDate < '20020501'  
GROUP BY CustomerID
```

107 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

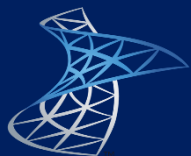
SELECT CustomerID, sum(SubTotal) CustomerTotal, count(*) OrderQt FROM dbo.SalesOrderHeader





- **Hands On:** Criando Índices Columnstore





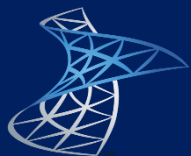
- **Alto consumo de memória na criação**

- ✓ Utiliza a memória do servidor no processo de criação.
- ✓ Quando o servidor não possui memória disponível retorna mensagem de erro.

```
The statement has been terminated.  
Msg 8658, Level 17, State 1, Line 1  
Cannot start the columnstore index build because it requires at least 341424 KB, while the  
maximum memory grant is limited to 189696 KB per query in workload group 'default' (2) and  
resource pool 'default' (2). Retry after modifying columnstore index to contain fewer  
columns, or after increasing the maximum memory grant limit with Resource Governor.
```

- ✓ Após a criação, persiste no disco e utiliza o cache do servidor do mesmo modo que índices Btree.
- ✓ Para estimar quantidade de memória:

$$\begin{aligned} \text{Memória em MB} &= ((4.2 * \text{Qtd colunas}) + 68) * \text{Qtd Threads} \\ &+ (\text{Qtd colunas string} * 34) \end{aligned}$$



• Alto consumo de memória na criação

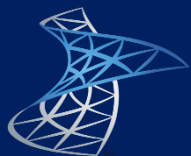
- ✓ Para reduzir a quantidade de memória necessária:
 - Reduzir a quantidade de colunas.
 - Reduzir a quantidade de Threads.
- ✓ Para reduzir quantidade de Threads utilizar parâmetro MAXDOP.

```
CREATE NONCLUSTERED COLUMNSTORE INDEX NCI_FactOnlineSales  
ON dbo.FactInternetSales (ProductKey,PromotionKey,SalesAmount,DiscountAmount)  
WITH (MAXDOP = 4)
```

Exemplo:

- Qtd colunas 21
- Uma coluna string
- Servidor com duas CPUs Quad-Core (8 Threads)

$$[(4.2 * 21) + 68] * 8 + (1 * 34) = 1.283,6 \text{ MB}$$



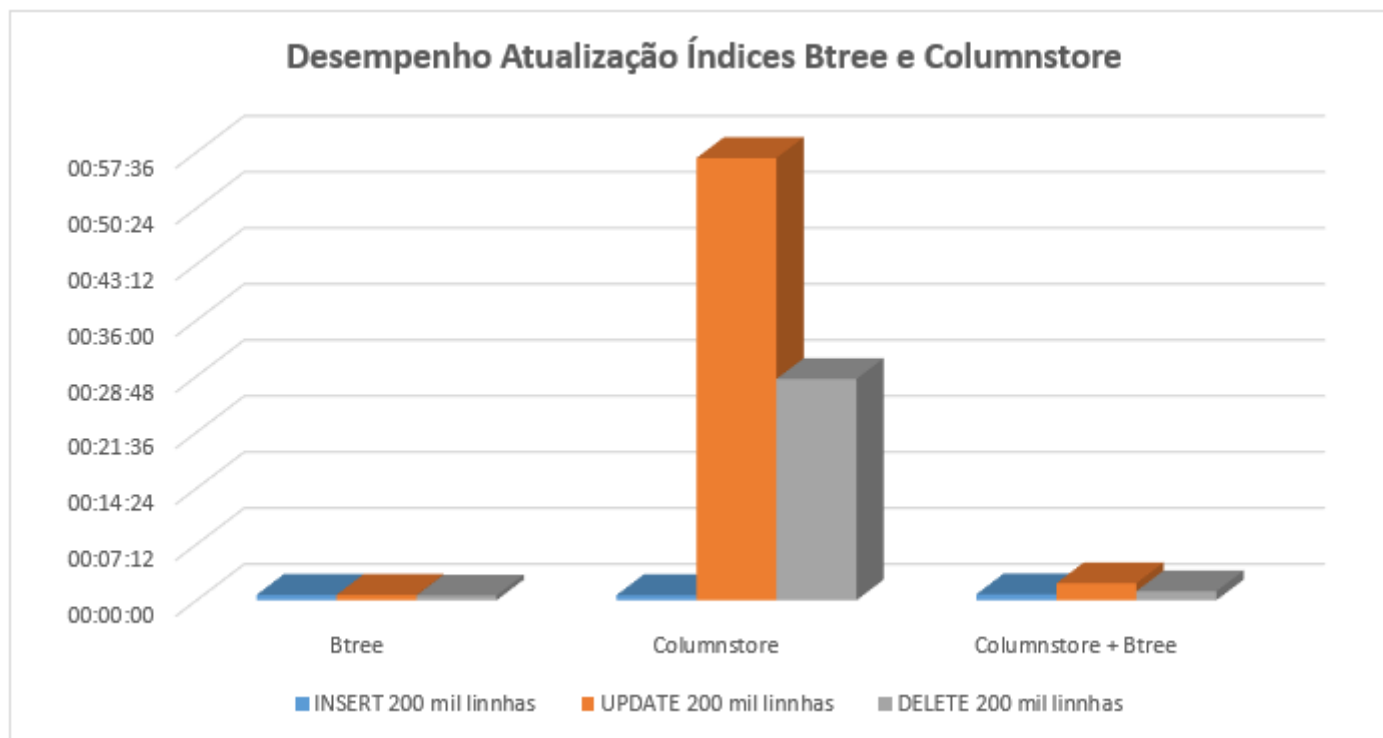
- **Limitações para uso do Índice Columnstore**

- ✓ **Maior consumo de memória do servidor.**
- ✓ **Baixo desempenho de atualização:** apesar das constantes melhorias a cada versão do SQL Server, tabelas com índices Columnstore apresentam desempenho de atualização inferior a tabelas sem este índice.
- ✓ **Incompatibilidade com alguns tipos de dados:** tipos de dados que não podem compor um índice Columnstore: ntext, text, image, rowversion (timestamp), sql_variant, XML e tipos de dados CLR (hierarchyid e spatial types). A partir do SQL Server 2017 nvarchar(max), varchar(max), and varbinary(max) podem ser utilizados.
- ✓ **Incompatível com Replicação.**
- ✓ **Incompatível com Filestream.**



- **Baixa performance de UPDATE e DELETE**

	Btree	Columnstore	Columnstore + Btree
INSERT 200 mil linhas	00:00:42	00:00:40	00:00:47
UPDATE 200 mil linhas	00:00:42	00:56:51	00:02:12
DELETE 200 mil linhas	00:00:38	00:28:27	00:01:10





- **Utilizar Índice Clustered Columnstore**

- ✓ Ideal para tabelas grandes em um DW, para Fatos ou Dimensão.
- ✓ Ideal também para tabelas com grandes volumes de INSERT.
- ✓ Grande compressão dos dados (10x), reduzindo o espaço ocupado.
- ✓ Melhor desempenho para consultas de análise.
- ✓ Poucas ocorrências de UPDATE e DELETE.

- **Não utilizar Índice Clustered Columnstore**

- ✓ Tabela possui varchar(max), nvarchar(max) ou varbinary(max).
- ✓ Menos de um milhão de linhas por partição.
- ✓ Mais de 10% das operações são UPDATE e/ou DELETE.